现在的问题主要是：1.教案排版丑，上传之前的教案可能排版还可以，但是因为有表格解析之后变成markdown格式影响观感

1.课堂模拟部分，有些教案读取不到课堂模拟的整个流程，怀疑是原教案原本是表格形式的，解析上传到系统之后暴力拆解为markdown导致表格格式出现问题，导致课堂模拟没有按照理想状况读取教学过程的各个模块，而是只读到表头或者把整个教学过程当做一个模块

2.协同共析部分，教案截取有些问题，比如下面图里这样，教学过程只读到表格，剩下的内容被搞到情境导入里面了。

感觉是数据间输入输出传递的问题，但是我没有头绪不知道怎么改



以下是ai分析的问题原因：

一、已确认的真实问题

1. V0 的 DOCX 解析其实是正常的

原始 DOCX 中"教学过程"是一张真实的 7×4 Word 表，解析后的 V0 也确实包含：

- 一个 `teaching_process` block；
- 6 个教学环节；
- 时长为 `5/12/8/9/4/2`；

- 每行的教师活动、学生活动、评价证据均存在；
- 原始 `tableRows` 也得到保留。

所以，不能继续把问题归因于“V0 一开始就没解析出来”。

2. F4 没有收到 V0 的结构化内容

`PlatformNavigationBridgeService` 创建 F4 会话时，只发送了：

```
content = version.content()
```

没有发送：

```
structuredContent
teachingProcess.rows
tableRows
```

F4 的 `CreateSession` 模型目前也只接受普通字符串 `content`。

因此，即使数据库里 V0 有完整的 6 行教学过程，进入 F4 后也只剩被压平的文本。

3. F4 对文本再次分段，丢失了 Word 表格语义

F4 的 `BoundedSectionParser` 本质上仍是标题和文本行分段器。

它能识别“教学过程”“情境导入”等标题，却不知道原来哪些内容属于同一张表中的哪一列。因此会出现：

- “教学过程”只剩表头；
- 具体表格内容被划到“情境导入”；
- 多个环节被合并；
- 教师活动、学生活动、评价证据失去对应关系。

这正是协同共析页面异常的直接原因。

4. F3 正式链路没有读取 F2 的 LessonDocument

F3 Runner 当前实际执行的是：

```
读取 F4 current-state
→ build_f3_materials(f4_state)
→ 把 F4 sections 转成 F3 materials
```

它没有读取 F2 数据库中的：

```
platform_lesson_version_structure.structured_content
```

所以 F3 看到的不是 V0 原本正确的六行结构，而是 F4 二次分段后的普通文本。

5. F4 → F3 Adapter 使用正则猜教学步骤

当前 Adapter 只有在普通文本同时满足下列条件时才会拆分：

- 存在“六、教学过程”；
- 有连续的 `1. 2. 3.` 步骤；
- 文本包含“教师活动”或“学生活动”等关键词；
- 至少识别出两个步骤。

否则整个“教学过程”会作为一个 material 发送给 F3。

因此才会出现课堂模拟先讨论：

```
环节与时间 | 教师活动 | 学生活动/可能反应 | 评价与证据
```

然后只进入“情境导入”，后续五个真实教学环节没有正常推进。

这不是 F3 Manager 自主结束逻辑的问题，而是它收到的材料就不完整。

6. V1 缺少结构记录

F4 保存新版本时，`PlatformNavigationBridgeService.persist()` 直接插入：

```
platform_lesson_version
```

但没有同步调用 LessonDocument 保存服务。

因此：

```
V1 content 存在
V1 structured_content 不存在
```

这与实际导出的 V1 结构文件为空完全一致。

7. V2 被主动降级成纯文本结构

F2 保存优化版本时，`PlatformLessonService.addVersionFromModule()` 会执行类似：

```
LessonDocumentSupport.fromPlainText(content)
```

于是 V2 被标记成：

```
FALLBACK
PLAIN_TEXT
```

V0 已正确解析的表格和教学过程结构没有传到 V2。

8. DOCX 导出没有使用 LessonDocument

当前导出逻辑主要是：

```
原生 F2 DOCX
或
基于 version.content 的 fallbackDocx()
```

fallback 导出器只把标准 Markdown 表格识别成 Word 表格，例如：

```
| 环节 | 教师活动 | 学生活动 |
|---|---|---|
```

而当前内容可能是：

```
环节 | 教师活动 | 学生活动
```

所以会被导出成普通段落，而不是真正的 Word Table。

二、最终结论

你前面提到的这些现象：

- F3 课堂模拟只拿到一个大环节；
- F4 教学过程拆错；
- V1 没有结构；
- V2 变成 `PLAIN_TEXT/FALLBACK`；
- 页面阅读体验差；
- 下载 Word 排版很丑；
- 原始 Word 表格消失；

确实可以归为同一类问题：

> LessonDocument 只在导入阶段存在，后续模块没有把它作为贯穿整个版本生命周期的主要数据结构。

三、建议的修复顺序

第一阶段：先修 F3，最快解决课堂模拟材料错误

修改 F2 → F3 正式链：

```
F2 当前 versionId
→ 读取 LessonDocument
→ teachingProcess.rows
→ 直接生成 F3 materials[]
→ F3 Session API
```

每个教学环节生成一个 material：

```
{
  "materialId": "稳定环节ID",
  "title": "情境导入",
  "sourceSectionId": "原始block/row标识",
  "teachingScript": "完整教师活动、学生活动和评价信息",
  "metadata": {
    "durationMinutes": 5,
    "teacherActivity": "...",
    "studentActivity": "...",
    "evaluation": "..."
  }
}
```

推荐实现方式：

- `PlatformSimulationLaunchService` 从当前版本结构生成 `materials`；
- 把 `materials` 写入 Runner 配置；
- `run_f3_demo.py` 如果配置中存在 `materials`，直接使用；
- 没有结构时才退回现有 F4 Adapter。

这样不需要改 F3 Manager、Agent、EventStore 或分析逻辑。

第二阶段：修 F4 的结构输入

F4 创建 Session 时应额外接收可选的结构信息，例如：

```
structuredContent
或
sections[]
```

F2 根据 LessonDocument 生成 F4 sections，至少保证每个教学过程环节拥有：

- 短标题；
- 时长；
- 教师活动；
- 学生活动；
- 评价证据；
- 原始 block/row 标识。

F4 原来的纯文本 `content` 仍保留，用于兼容旧调用，不要直接删除。

第三阶段：修版本结构继承

需要明确处理：

- F4 写回生成 V1 时保存 `structured_content`；
- F2 优化生成 V2 时不能无条件调用 `fromPlainText()`；
- 可以结构化解析时保存正式 LessonDocument；
- 只有确认无法恢复结构时才允许 `FALLBACK/PLAIN_TEXT`；
- 每个版本都应该有对应的 `platform_lesson_version_structure`。

第四阶段：重做正式 DOCX 导出

导出时使用：

```
lessonId + versionId
→ LessonDocument
→ Word 模板
→ 当前版本 DOCX
```

教学过程必须生成真实 Word 表格，而不是 Markdown 字符。

如果当前版本没有结构，才使用现有纯文本 fallback，并给出结构降级提示。

四、能否支持“所有格式”

不能承诺任何 DOCX 都能 100% 自动识别。合理目标应该是：

文档情况	系统处理
标准四列表格	自动识别
三列表格	通过表头语义映射
列名同义词	通过同义词字典识别
合并单元格	展开或继承上方字段
多张连续教学过程表	合并为同一教学过程
普通编号段落	按编号和时长识别
完全自由文本	进入人工确认/降级模式
无法可靠判断	保留原文，不虚构结构

重点不是写更多针对某一份教案的正则，而是建立：

```
DOCX 原始结构
→ 规范化中间结构
→ 教学语义识别
→ 质量检查
→ 必要时人工确认
```